

Table Football Feedback System



Navid Javid

Gazal Ghashghaei

Yazdan Haghi

Presentation Structure

Main Sections

This presentation covers the full system from hardware to software:

- Project Overview
- Online & Offline Operation
- Embedded Firmware & Software
- Dashboard & Admin Panel
- Physical Components & Dual Boards
- Game Modes: 1v1, 2v2 & Tournaments
- Hardware Design & PCB
- Remaining Future Work

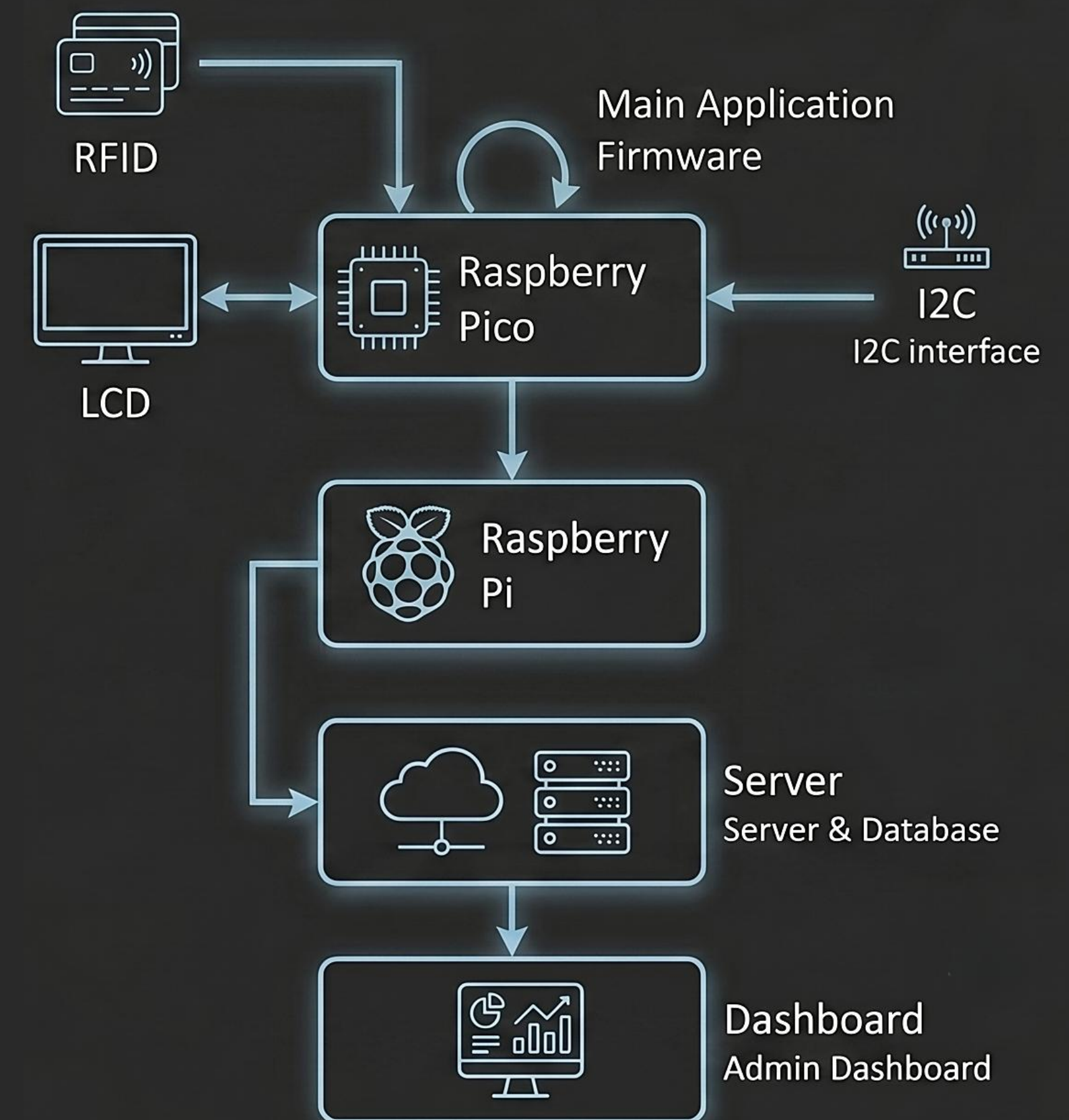


What the Project Does

This project turns a normal table-football setup into a **smart feedback system**.

Each player identifies themselves by tapping an **RFID/NFC card** on their side. Each side's **Pico** reads the player identity, shares ball and score data with its sibling board over **I2C**, updates its local **LCD**, and sends live match information to a **Raspberry Pi** server.

The Raspberry Pi stores player and match data, manages the live dashboard, and allows users to view scores, player profiles, and game history.



System Architecture

How the Main Parts Work Together

Layer 1

Device Layer

Two independent side-PCBs (one primary, one secondary) each read their own RFID reader and a shared ball-position feed, and coordinate directly with each other over I2C when the Pi is unreachable

Layer 2

Communication & Backend Layer

WiFi + MQTT carry table events to the Raspberry Pi, where server logic is the single source of truth for game state.

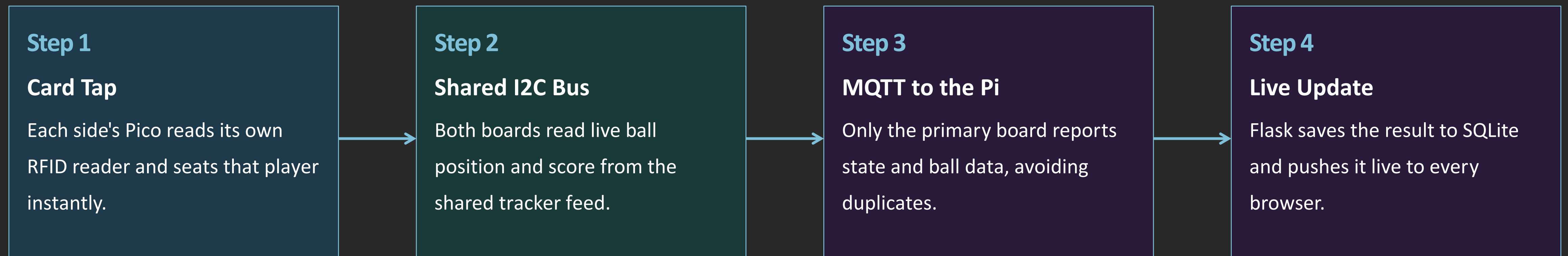
Layer 3

Data & Presentation Layer

SQLite stores permanent match history while Flask + SSE push live updates to dashboards, profiles, and the admin panel.

System Operation — Online Mode

End-to-End Data Flow (Wi-Fi + MQTT)



Server as source of truth: while online, the Pi's database is authoritative for match history and player stats — the boards' own local game state is just the real-time display.

Offline Mode & I2C Data Exchange

Works With No Wi-Fi or Server

- **Card Tap (Offline)**

Each side's Pico reads its own RFID reader and registers the player locally

- **Shared I2C Bus**

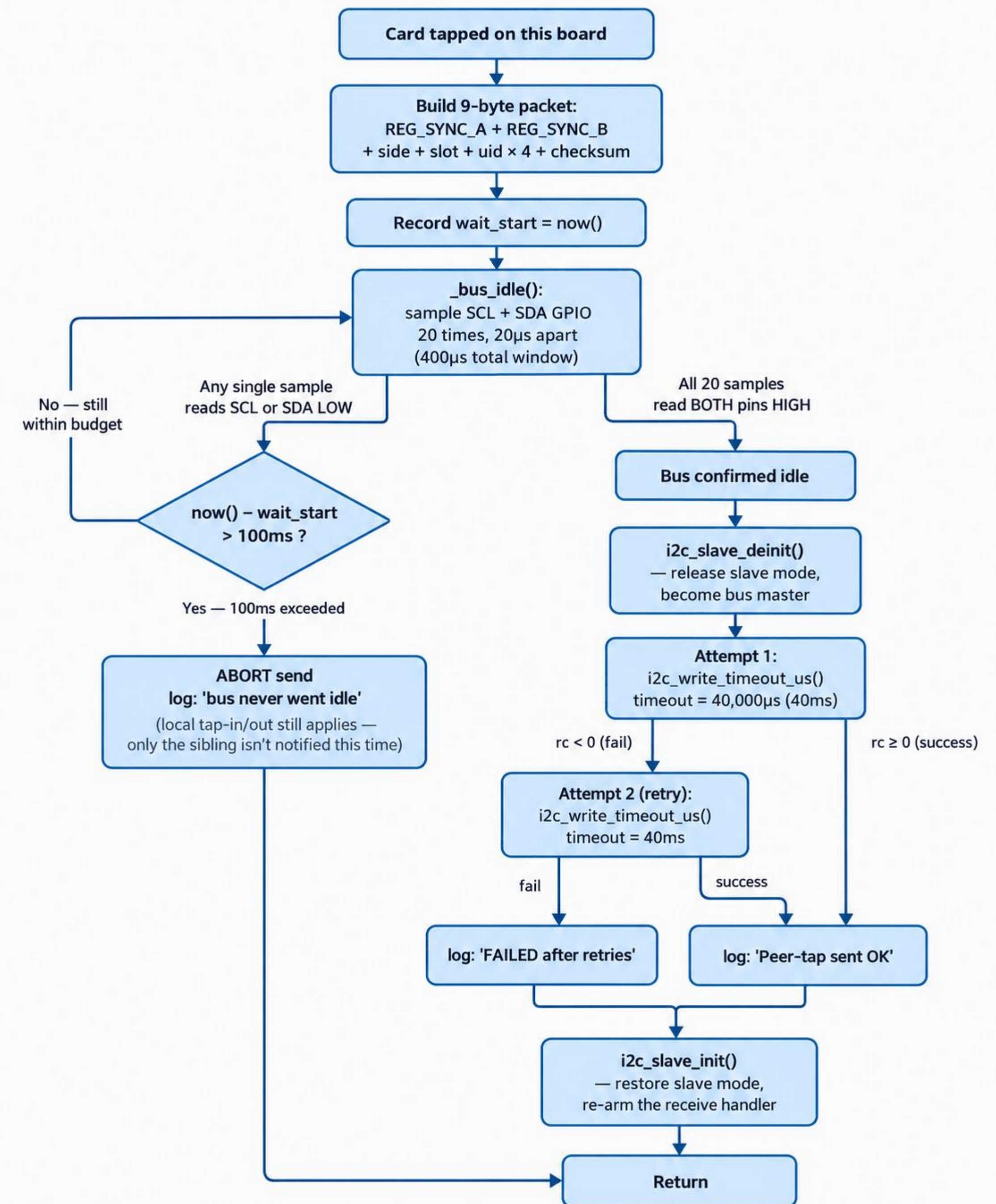
The ball tracker and the two boards' own tap messages share the same wires; a board waits for a quiet moment on the bus before telling its sibling about a tap.

- **Local Game State**

Both boards track score, goals, and match-over entirely on-device — every decision is made locally, with no Pi involved at all.

- **Local Name Directory**

Player names come from a directory synced earlier from the Pi and saved on the board's own flash . Real names still show up offline, even after a power cycle.



Embedded Firmware Structure

Shared by Both Boards

`main.c`

The coordinator. Initializes Wi-Fi, MQTT, I2C, RFID, display, and game logic. Runs the main loop.

`rfid_handler.c`

Works with the PN532 driver and handles card tap detection for player identification.

`i2c_comms.c`

Receives ball data and relays peer-tap messages over the same shared I2C bus.

`game_logic.c`

Manages player registration, score updates, fastest-shot tracking, and game state transitions.

`display_manager.c`

Converts the current game state into screens for the LCD display.

`pico_mqtt.c`

Publishes live data to the Pi and applies admin commands (identify, reset, message) sent back.

One firmware image runs on both boards — a single compile-time flag decides which side and role a board takes on.

Component Libraries

Low-Level Drivers



PN532 Library

Handles SPI communication with the NFC reader. Sends PN532 command frames, waits for responses, and returns detected card UIDs to the RFID handler.



EA DOGL128 Library

Manages the LCD framebuffer, drawing functions, text rendering, rectangles, lines, contrast control, and screen flushing.

Key Benefit: This separation is useful because the main game logic does not need to know the electrical details of the display or RFID reader. It only calls higher-level functions like *"scan card"* or *"draw game screen."*

Main Firmware Loop

How the Pico Runs the System

1 Keep Connections Alive

Poll the WiFi/lwIP stack every loop iteration so the WiFi link and MQTT connection stay serviced.

2 Check I2C Ball Data

Update local game state from the shared feed; only the primary board forwards it to the Pi.

3 Check RFID Reader

If a new UID is detected, change player registration and publish an RFID event.

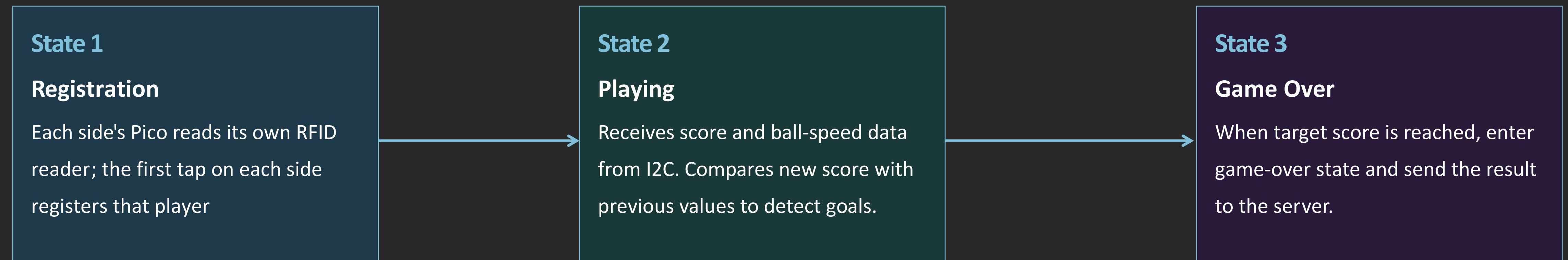
4 Refresh & Publish

Refresh the LCD and publish MQTT messages at controlled intervals. heartbeat every 5s, ball position ~12 times a second.



Game Logic

From Player Registration to Game Over



Works in 2v2 too: a side can hold two players; tapping an already-seated card again removes that player — even mid-match, which safely ends the match if a side empties.

Communication Between Components

Connection Types



SPI — PN532 Reader

RFID communication needs reliable short hardware transactions between the Pico and the reader. SPI provides fast, synchronous data transfer for card detection.



Bit-Banged SPI — LCD

A bit-banged SPI-style interface gives more reliable timing with the specific EA DOGL128L-6 display module used in the prototype.



I2C — Ball Data

receives live ball position, speed, and score. In the final project, this comes from a camera-based tracking system over I2C



Wi-Fi + MQTT — Pico to Pi

MQTT is lightweight and supports structured topics for heartbeat, RFID events, game state, and ball data. Ideal for live embedded updates.

Raspberry Pi Server Stack

Server-Side Architecture



MQTT Broker (Mosquitto)

Receives all published messages from the Pico — heartbeat, RFID events, game state, ball data.



Flask Web Application

Provides the public dashboard and admin pages. Serves dynamic content to connected browsers.



SQLite Database

Stores players, matches, live table state, match players, and Pico device status.



Server-Sent Events (SSE)

Pushes live changes from the server to browsers in real time without polling.

MQTT Communication

Pico to Server Data Exchange

MQTT is used because it is **lightweight** and **reliable** for live embedded data. The topic structure makes it easy to scale.

Topic Structure

	QoS Level
tablefootball/pico/<id>/ heartbeat — Pico online status	0
tablefootball/table/<id>/ rfid — Card tap events	1
tablefootball/table/<id>/ state — Score, winner, status	1
tablefootball/table/<id>/ ball — Live pitch position data	0

Scalable Design: This topic hierarchy makes it easy to add more tables and Picos without changing the server architecture.

MQTT Quality of Service (QoS)

MQTT lets every publish choose a delivery guarantee — trade reliability for speed/overhead.

QoS 0 — "Fire and forget"

Message sent once, no acknowledgment, no retry Fastest, lowest overhead, but a message can be lost.

QoS 1 — "At least once"

Receiver must acknowledge; sender retries until it does Guarantees delivery, but the same message can arrive more than once.

QoS 2 — "Exactly once"

Four-step handshake guarantees delivery with no duplicates Most reliable, but slowest and heaviest. not used in this project

Public Dashboard

Live Match View

- **Live Match Cards**
Each table displayed as a live match card with player names, guest or registered badges, score, game status, fastest shot, and winner.
- **Mini Pitch Visualization**
Displays the ball position in real time using the received I2C data.
Spectators can follow the action visually.
- **Real-Time Updates**
Users connect to the Raspberry Pi hotspot and watch the match from any phone or laptop browser. No app installation needed.

LIVE TABLES

Table 1

Alice

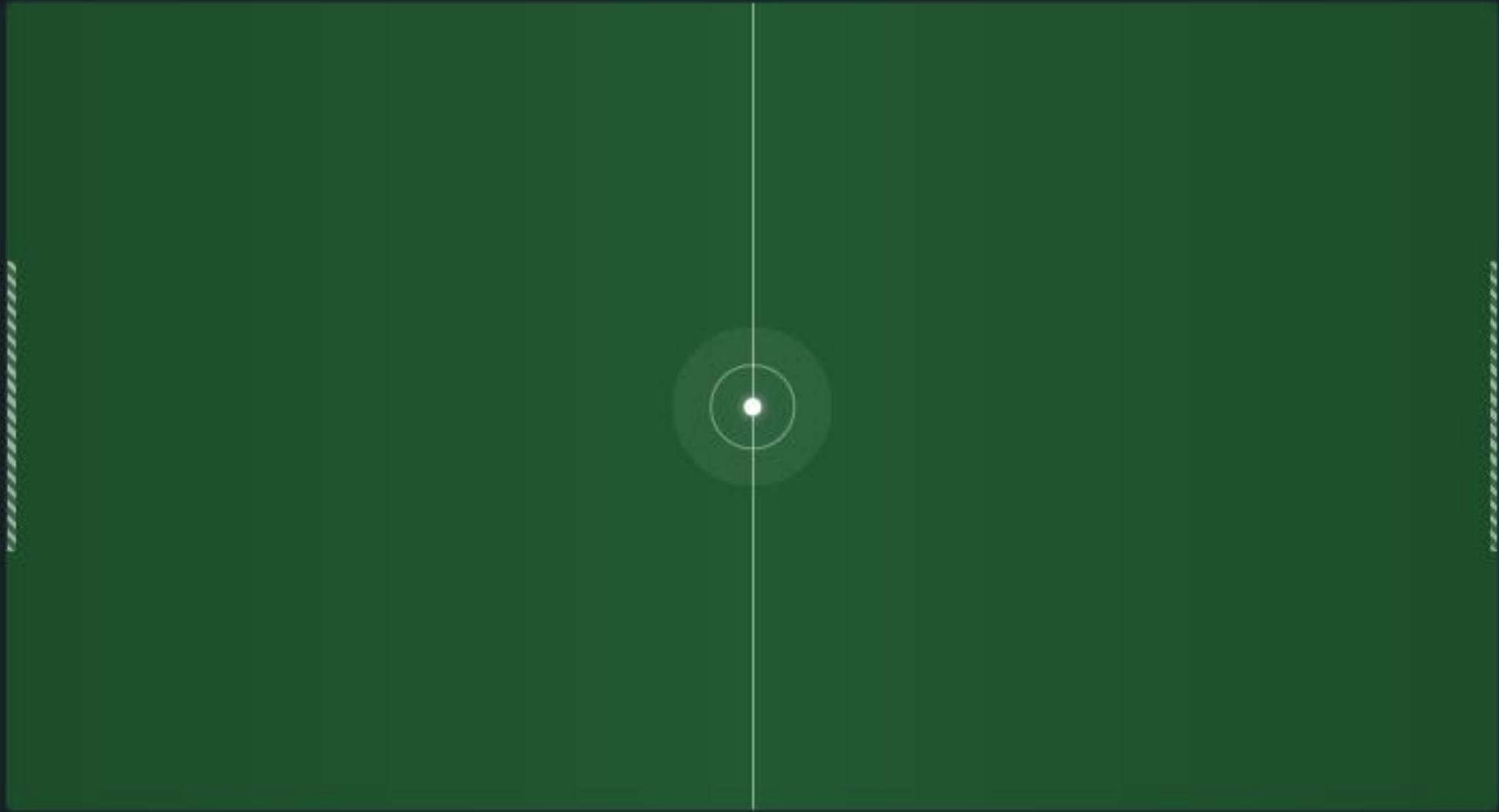
1

:

Bob

0

PLAYING



Mode: 1v1

Best: 16.9 km/h

Max: 10

ALL PLAYERS

PLAYER	RFID UID	GAMES	WINS	LOSSES	BEST SHOT	STATUS
Alice REGISTERED	DBEF7B05	4	3	1	23.1 km/h	Active
Bob REGISTERED	4C869804	3	0	3	23.1 km/h	Active
Carol REGISTERED	DB4E6C05	2	1	1	23.1 km/h	Offline
Dave REGISTERED	C2878704	4	0	4	23.1 km/h	Offline

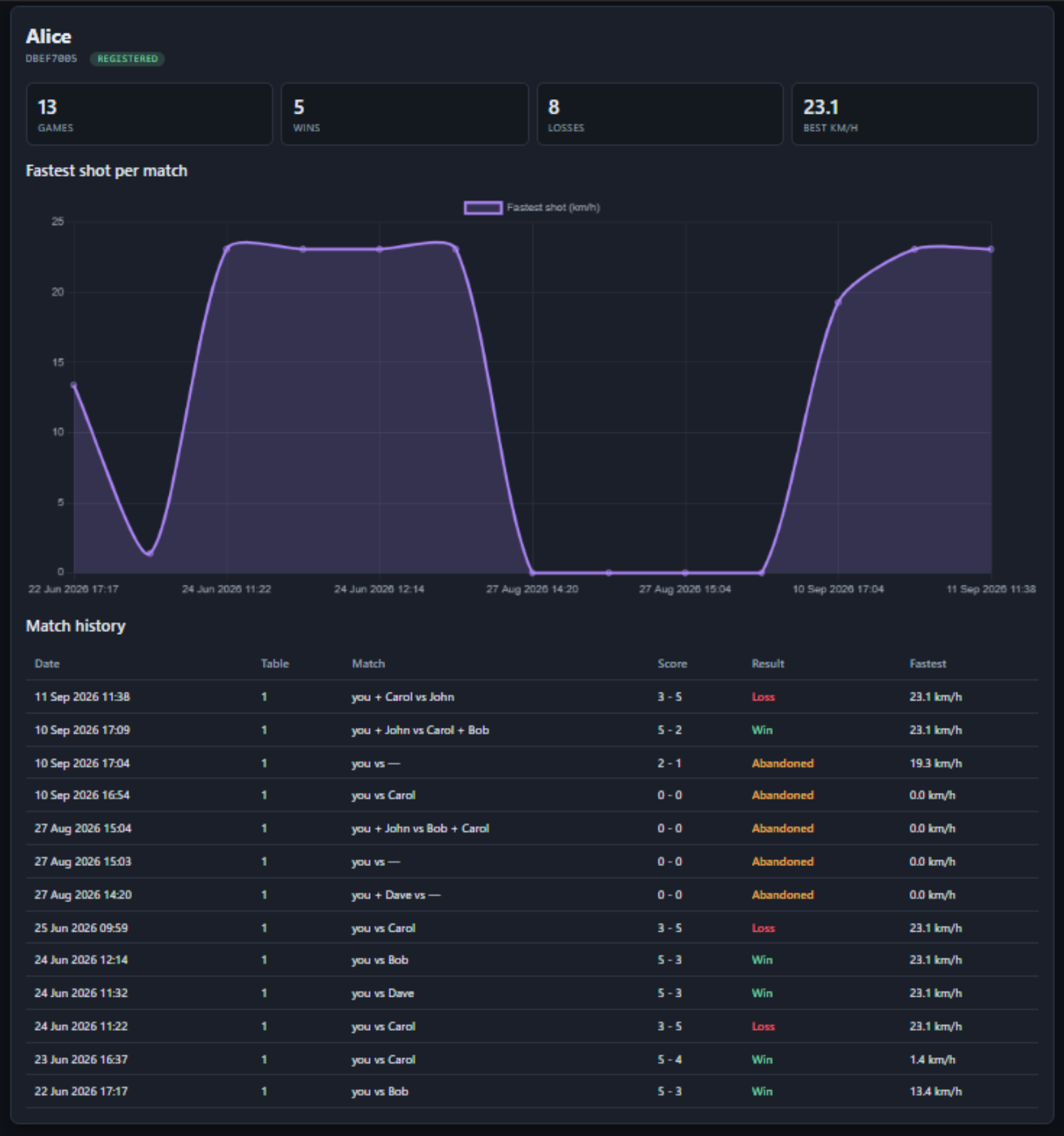
Admin Panel and Player Profiles

Managing Players and Devices

Admin Panel Features

- **Player Management:** Rename guest players into registered ones once you know who they are..
- **Pico status monitor & remote control:** see which Picos are online/offline, and send Identify, Reset, Clear Players, or a Message to a board.
- **Table assignment:** assign a pico_id to a table_id if a Pico hasn't been linked yet.
- **Table & tournament management :** create tables and tournaments, generate brackets, assign matches to tables, and start/stop/add/remove players without a card tap.
- **Authenticated access:** protected by a login screen, password set via .env.

Player Profile Stats



2v2 & Tournament Mode

Now Fully Implemented

01 Real 2v2 Matches

Either side can hold up to two players; the mode label (1v1/2v2/mixed) is computed automatically from who's seated and shown live on the dashboard.

02 Single-Elimination Brackets

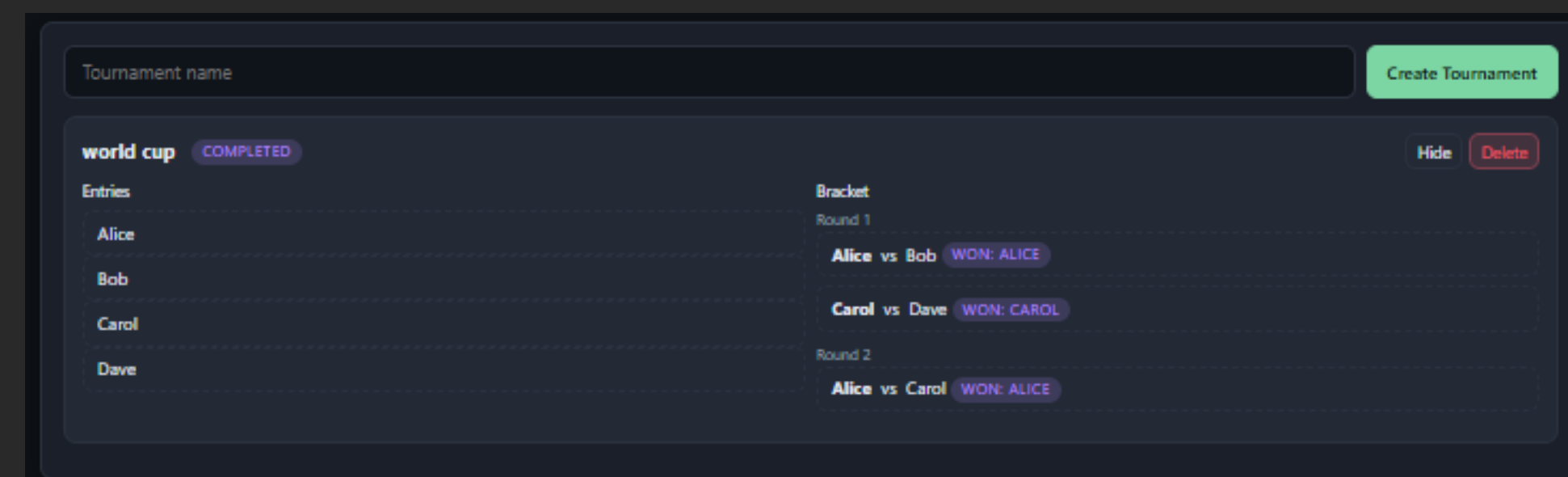
An admin creates a tournament, adds entries, and generates a full single-elimination bracket in one click — byes are handled automatically.

03 Assign a Match to a Table

Assigning a pending bracket match seats its two entries directly and starts the game — no card tap required to begin.

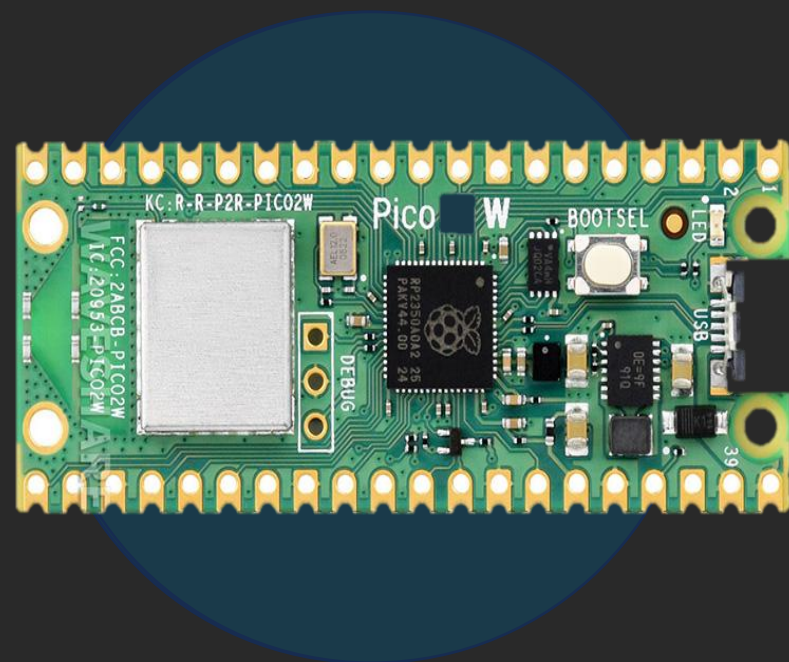
04 Automatic Advancement

The winner of a real, completed match on the assigned table is recorded straight into the bracket, and the next round becomes available automatically.



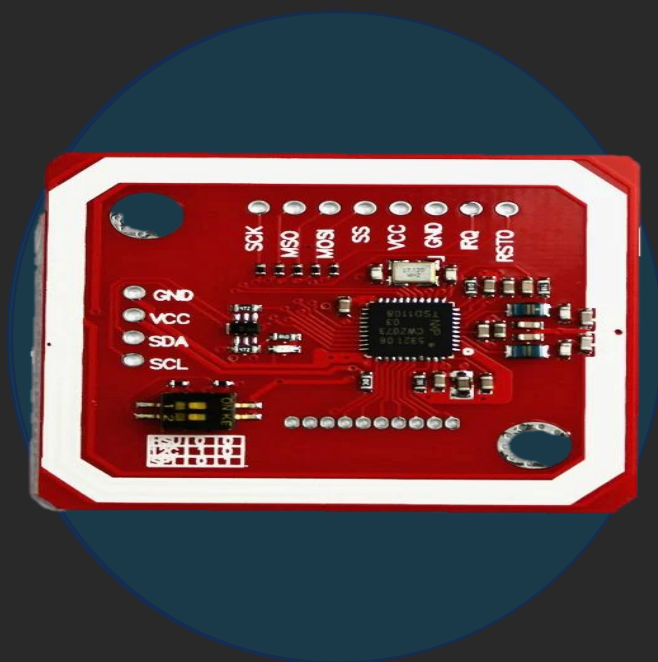
Physical Components

Main Hardware Used



Pico 2 W

Main embedded controller with Wi-Fi connectivity



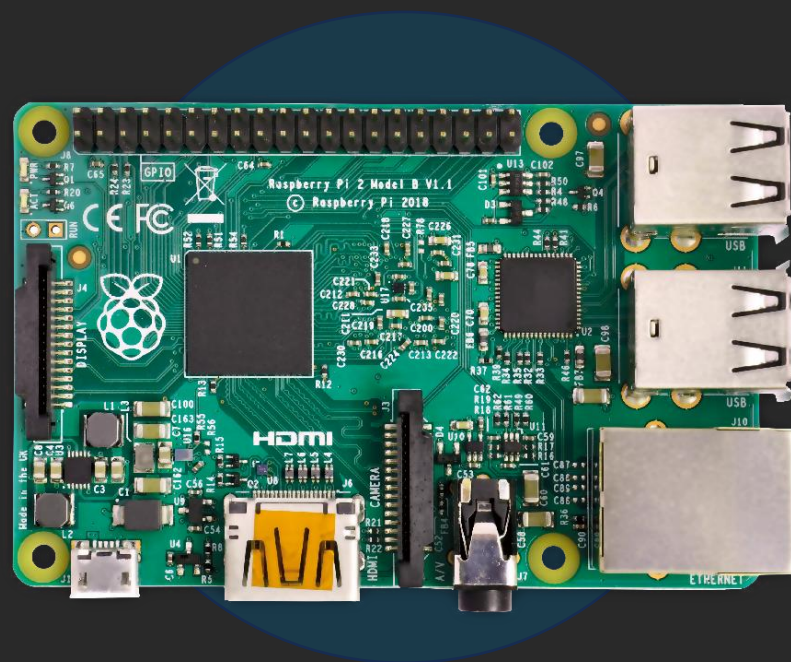
PN532 Reader

NFC/RFID card reader for player identification



EA DOGL128L-6 LCD

Graphic LCD for local score and status display



Raspberry Pi 4

Central server, MQTT broker, and web dashboard

Component Roles

Physical Integration

- **PN532 — Player Identification**

Reads RFID/NFC cards. Each card UID is treated as a unique player identity.

- **EA DOGL128L-6 — Local Feedback**

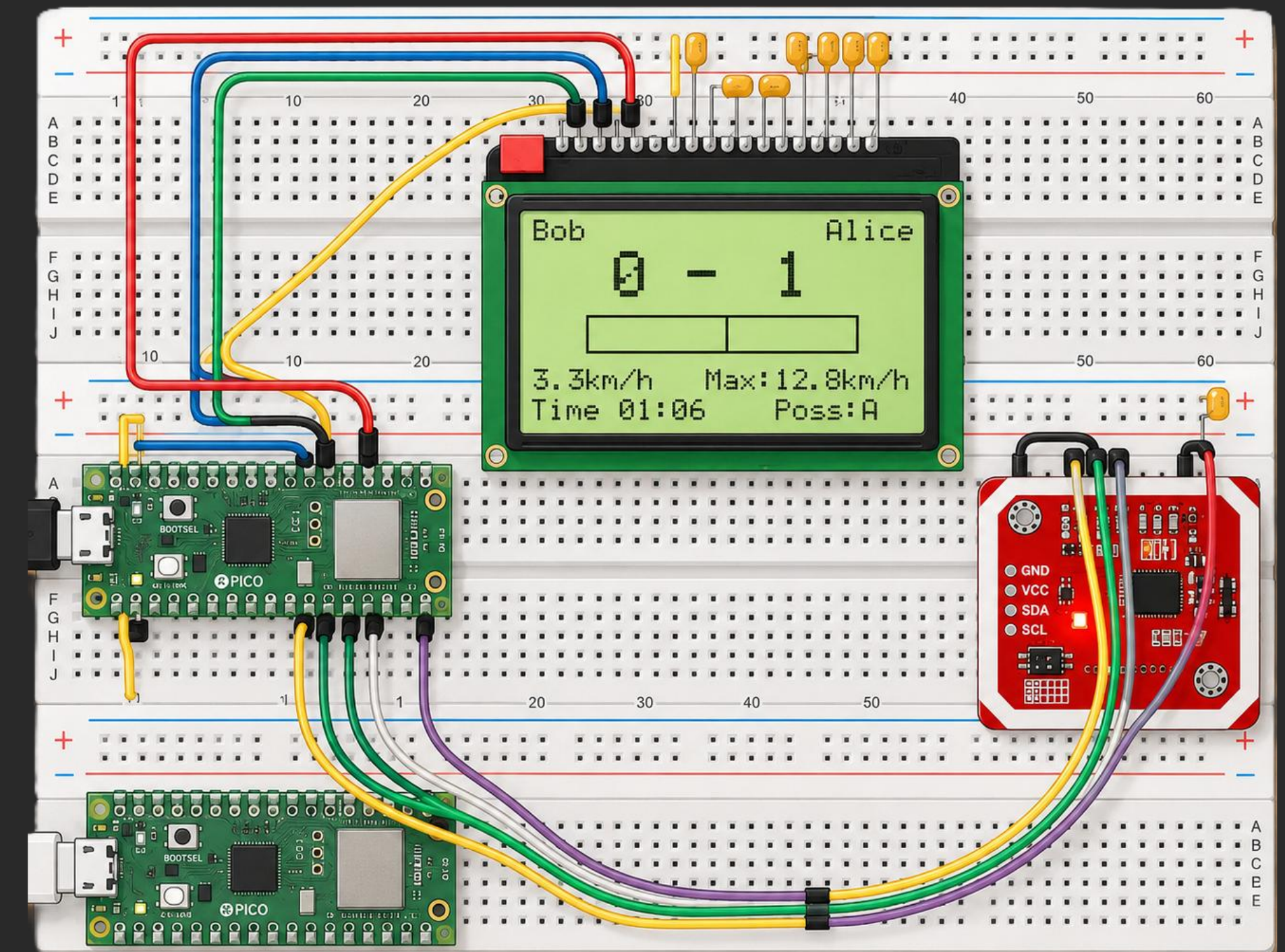
Gives immediate feedback: registration state, player names, score, goal animation, and game-over status.

- **I2C Connection — Ball Data**

the wired link that carries live ball-position and score data from the camera to the Pico

- **Raspberry Pi 4 — Central Server**

Dashboard hosting, data storage, and player management hub.



Hardware Deep-Dive: Custom PCB

From Breadboard to Fabricated Board

- **Two-Layer Design**

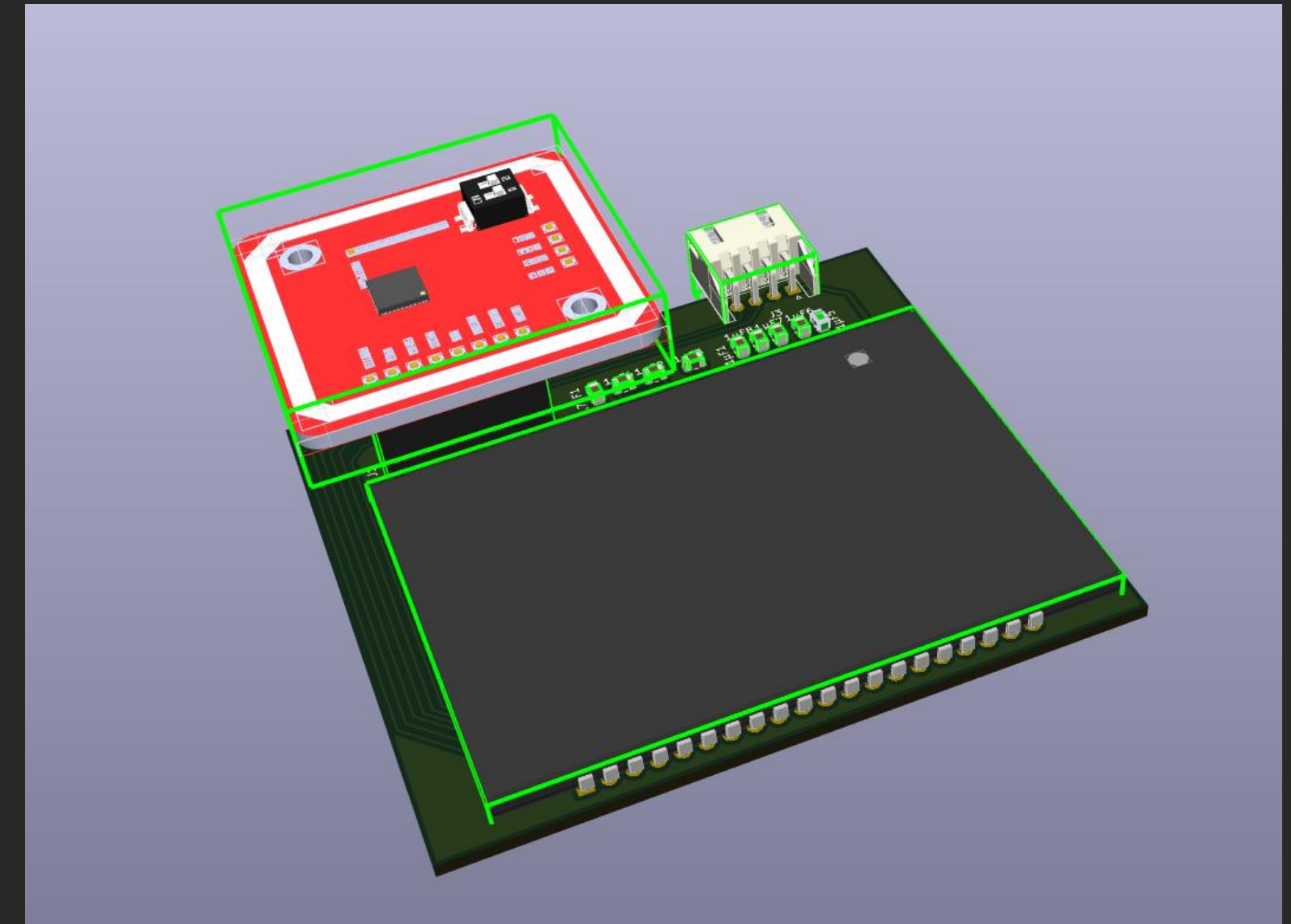
The board integrates the Pico 2 W, PN532 reader, and LCD header onto one routed PCB, replacing the original breadboard wiring entirely.

- **Decoupling & Power**

Eight $1\mu\text{F}$ ceramic capacitors and a $4.7\mu\text{F}$ bulk capacitor sit right next to the display header, feeding its charge-pump cleanly.

- **I2C Header for the Shared Bus**

A dedicated 4-pin connector breaks out SDA/SCL/power to the shared ball-tracker bus, keeping wiring identical on both side-boards.



Local LCD User Interface

Feedback on the table, no dashboard needed

The LCD gives players immediate feedback at the table — score, ball position, and match result all update live, with no need to look at a phone or the dashboard.



Registration

Prompts each side to tap a card. The prompt blinks until both players are registered, then play starts automatically.

Match Playing

Shows names, live score, a mini field with the ball's position, current and fastest shot speed, match timer, and possession.

Goal & Game Over

An animated goal screen pops up the instant a side scores. At match end: winner, final score, fastest shot, and a prompt to tap for a new game.

Physical Enclosure & Deployment

From Prototype to Cabinet-Ready

- **3D-Printed Enclosure**

A enclosed box with a separate front door houses the board, display, and RFID reader, and can be mounted directly onto the table football structure

- **Friendly Local URL**

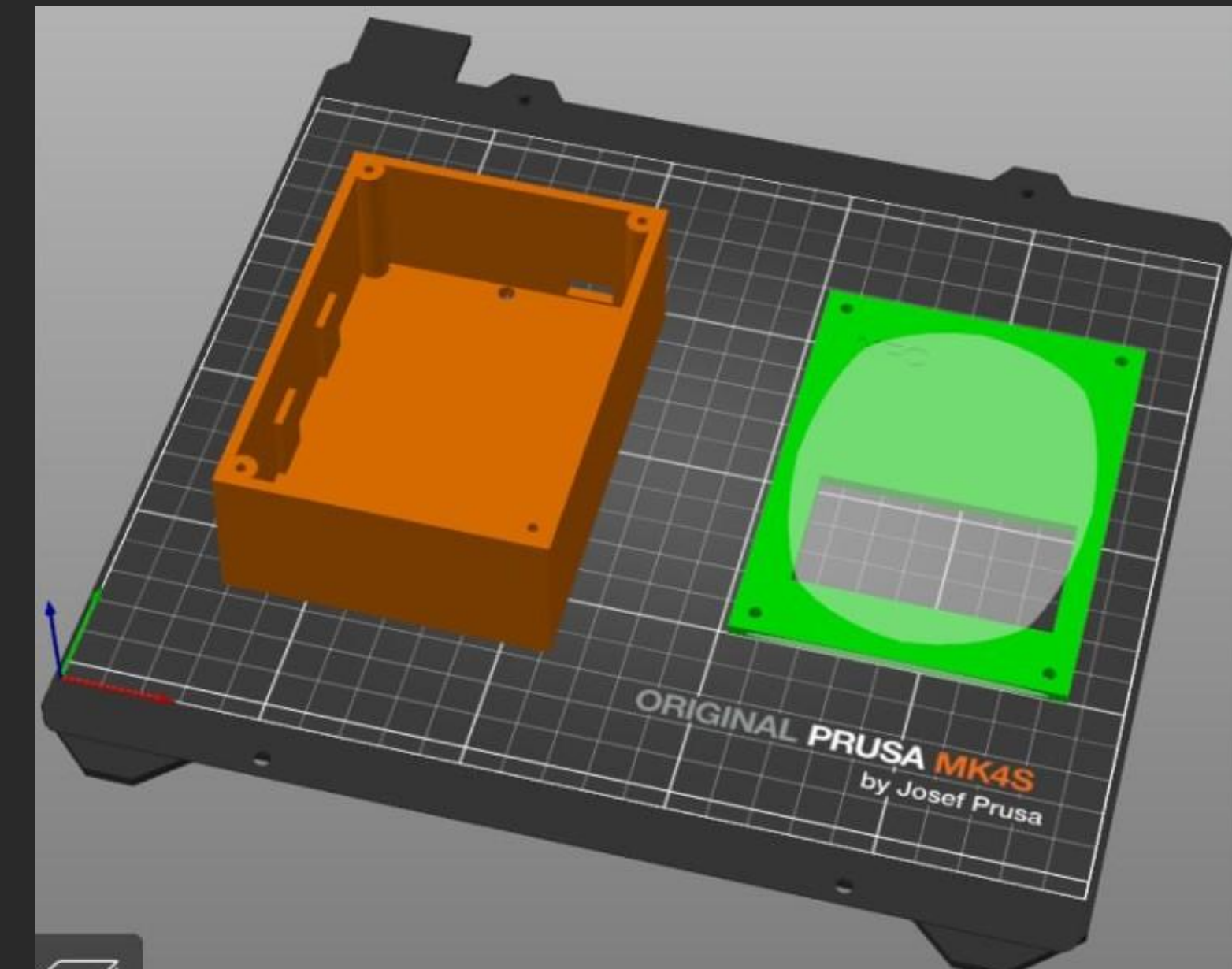
The dashboard is reachable at a plain tb.local with no IP address or port number to remember, over the table's own Wi-Fi hotspot.

- **Managed as a System Service**

The Flask server runs under systemd with automatic restart on failure and a scoped network capability, instead of a manually-started script.

- **Remote Board Control**

Admins can identify, reset, or message a specific Pico from the dashboard without walking over to the cabinet.



Remaining Future Work

Honest Next Steps

01

Offline Match-History Sync

Offline matches aren't written back to the Pi's database once it reconnects.

02

Self-Service Player Registration

New players need an admin to register them once, instead of a simple first tap.

03

Dedicated Pico-to-Pico Link

A dedicated UART or Wi-Fi link between the two boards would remove the current I2C bus-sharing risk entirely.

Thank You Questions?

Navid Javid

Gazal Ghashghaei

Yazdan Haghi